

System Design

Ticket Booking System — Seat Holds, Concurrency, Waitlist & Offer Handling

1 Seat Hold Mechanism

The seat hold is the most critical mechanism in the system. When a customer selects seats, we must guarantee that exactly one person can hold any given seat at a time, even under high concurrency.

Transaction Flow

1 Row-Level Locking

Inside a Prisma \$transaction, we execute `SELECT ... FOR UPDATE` on the target ShowSeat rows. This acquires exclusive row-level locks in PostgreSQL, serializing all concurrent hold attempts for the same seat(s). Any competing transaction blocks until the lock is released.

2 Lazy Expiry Check

Before trusting a seat's status field, we check `holdExpiresAt` against the current time. If a seat is marked HELD but its hold has expired, we treat it as AVAILABLE and update it in the same transaction. This pattern is applied everywhere seat status is read: the seat map endpoint, the hold endpoint, and the checkout endpoint.

3 State Update

Valid seats are updated to HELD with `heldById` (the requesting user) and `holdExpiresAt` (current time + configurable TTL, default 10 minutes).

4 Idempotency

If the user already holds the seat and the hold has not expired, the request succeeds without error or extending the hold.

Database-Level Safety Net

Prisma does not support partial unique indexes natively, so we add one via raw SQL migration:

```
CREATE UNIQUE INDEX "ShowSeat_active_idx"
ON "show_seats" ("showId", "seatId")
WHERE status IN ('HELD', 'BOOKED');
```

This is a belt-and-suspenders guarantee: even if the application-level lock has a bug, PostgreSQL rejects any attempt to create a second active hold or booking on the same seat.

Why Lazy Expiry Exists

The backend runs on Render's free tier, which sleeps after 15 minutes of inactivity. When the backend sleeps, BullMQ delayed jobs cannot fire. Without lazy expiry, a seat held at 8:00 PM with a 10-minute TTL would remain marked HELD indefinitely if the worker never wakes. Lazy expiry ensures correctness is **never dependent on job timing** — every code path that reads seat status dynamically evaluates the true state using timestamps. BullMQ jobs exist purely as an optimization for proactive cleanup and real-time Socket.io broadcasts.

2 Concurrency Prevention

SELECT ... FOR UPDATE with Serializable transaction isolation is the primary mechanism. Combined with the partial unique index, we achieve two independent layers of protection:

Application Layer	Database Layer
Row locks serialize concurrent transactions	Partial unique index prevents double-holds at the storage level

The concurrency test fires 20 simultaneous hold requests for the same seat and asserts exactly one succeeds with 19 clean rejections.

3 Waitlist Auto-Assignment

Assignment Flow

- 1

Cancellation Trigger
When a booking is cancelled, the system finds the freed seat's category and checks the waitlist.
- 2

Concurrent-Safe Selection
Using SELECT ... FOR UPDATE SKIP LOCKED, we find the oldest WAITING entry for that show and category. SKIP LOCKED prevents deadlocks when two cancellations happen simultaneously — each pulls the next distinct waitlist entry.
- 3

Offer Creation
The seat is set to OFFERED, the waitlist entry to OFFERED with offerExpiresAt, and a BullMQ delayed job is scheduled.
- 4

Notification
An email with a signed, time-limited link is sent. If the customer is online, a Socket.io event is also emitted.

4 Time-Limited Offer Handling

Cascading Expiry

When an offer expires (via the BullMQ delayed job or lazy expiry check on the next access):

- 1

Mark Expired
The waitlist entry is marked EXPIRED.
- 2

Re-Invoke Waitlist
The system immediately re-invokes waitlist processing for the same seat, pulling the next WAITING entry.
- 3

Cascade or Release
This cascade continues until either a waitlist entry is successfully offered or no more entries remain (seat becomes AVAILABLE).

Link Security

The signed offer link uses HMAC-SHA256 with the HMAC_SECRET environment variable, encoding the waitlistId and offerExpiresAt timestamp. The server verifies both the signature and the timestamp before allowing checkout, applying the same lazy-expiry pattern.

5 Free-Tier Hosting Constraints

Service	Constraint	Mitigation
Render (backend)	Sleeps after 15 min idle	Lazy expiry on all seat/offer checks
Upstash Redis	10K commands/day free	Minimal Redis usage (BullMQ jobs only)
Neon (PostgreSQL)	Compute suspends after 5 min idle	Connection pooling via Prisma
Gmail SMTP	Daily send limits	Email failures never roll back bookings

Correctness is preserved despite all constraints because the system **never relies on background jobs for state consistency** — they are purely an optimization for proactive cleanup and real-time broadcasts.